"Sapienza" Università di Roma
Ingegneria dell'Informazione, Informatica e Statistica
Dipartimento di Informatica

Linguaggi e Compilatori

Autore
Vincenzo Bova

A.A. 2025/2026

Indice

1 Introduzione **1**
1.1 Linguaggi Compilati vs Linguaggi Interpretati 1
1.2 Struttura di un compilatore 2
1.3 Scoping 2
1.4 Passaggio dei parametri 2

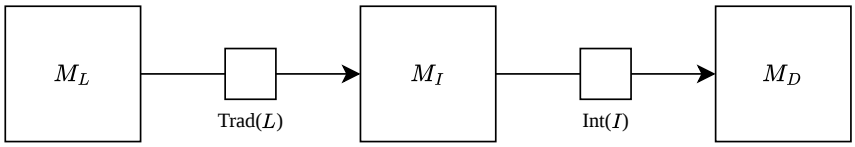
2 Costruzione di un compilatore **3**
2.1 Sintassi 3
2.2 Traduzione 6
2.3 Parsing 9
2.4 Parsing + Traduzione 12

1
Introduzione

1.1 Linguaggi Compilati vs Linguaggi Interpretati

In informatica moderna, quasi nessun linguaggio è puramente compilato oppure puramente interpretato. Il codice sorgente non viene mai passato direttamente all'hardware, ma esiste una gerarchia di componenti che collaborano tra loro:

- **Macchina del linguaggio (M_L):** è l'entità in grado di "capire" direttamente programmi in linguaggio L;
- **Trad(L):** prende il codice analizzato da M_L e lo traduce nel linguaggio I;
- **Macchina intermedia (M_I):** è l'entità in grado di "capire" il linguaggio intermedio I;
- **Int(I):** prende il codice analizzato da M_I e lo traduce nel linguaggio D;
- **Macchina di destinazione (M_D):** un tempo era la macchina fisica M_{HW} e D era il set di istruzioni della sua CPU. Con l'avvento dei Sistemi Operativi è diventata M_{SO} e D è composto da linguaggio macchina e chiamate di sistema.

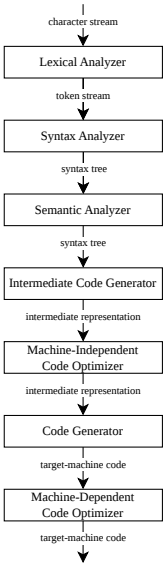


Nella realtà il 99% dei linguaggi ad alto livello hanno M_I significative e i linguaggi puramente compilati o puramente interpretati sono dei casi limite.

1.2 Struttura di un compilatore

Un compilatore può essere suddiviso in due grandi blocchi:

- **Frontend:** si occupa di analizzare il codice sorgente e assicurarsi che sia corretto ed è composto da: *Lexical Analyzer*, *Syntax Analyzer* e *Semantic Analyzer*;
- **Backend:** si occupa di tradurre il programma analizzato in codice eseguibile ed è composto da: *Intermediate Code Generator*, *Machine-Independent Code Optimizer*, *Code Generator* e *Machine-Dependent Code Optimizer*.



In particolare, *Lexical* e *Syntax Analyzers* non lavorano quasi mai in sequenza lineare ma collaborano continuamente, mentre *Semantic Analyzer* e *Intermediate Code Generator* sono spesso fusi assieme. Ogni stadio della compilazione riceve e produce una rappresentazione intermedia, modificando la **Symbol Table**.

1.3 Scoping

Un compito fondamentale del compilatore è gestire la visibilità degli identificatori (variabili, funzioni, etc.) tramite le regole di **scoping**:

- **Scoping statico:** la visibilità di un nome è determinata al tempo di compilazione in base alla struttura del testo;
- **Scoping dinamico:** la visibilità dipende dall'ordine di chiamata delle funzioni a tempo di esecuzione.

1.4 Passaggio dei parametri

Determina come le informazioni vengono trasmesse tra il chiamante e la funzione:

- **Call by value:** viene passata una copia del valore. È sicuro ma può essere inefficiente per strutture grandi;
- **Call by reference:** viene passata la locazione di memoria. Permette effetti collaterali e risparmio di memoria;
- **Call by name:** l'espressione viene valutata solo quando effettivamente usata nella funzione (tipico dei linguaggi funzionali).

2
Costruzione di un compilatore

2.1 Sintassi

Il primo passo per costruire un compilatore prevede di definire formalmente la grammatica del linguaggio.

2.1.1 Grammatiche Context-Free (CFG)

Definizione 2.1: Grammatica Context-Free

Una grammatica context-free è una quadrupla $G = (T, N, P, S)$, dove:

- **Insieme di simboli terminali (T):** simboli elementari non sostituibili;
- **Insieme di simboli non terminali (N):** variabili sintattiche sostituibili tramite produzioni;
- **Insieme di produzioni (P):** regole di riscrittura nella forma $A \to \alpha$;
- **Simbolo di partenza (S):** il punto di inizio della derivazione, con $S \in N$.

A questo punto, data una stringa contenente almeno un non terminale, possiamo scegliere un non terminale contenuto al suo interno e sostituirlo mediante una delle produzioni definite nella grammatica.

Definizione 2.2: Derivazione

Data una CFG $G = (T, N, P, S)$, una **derivazione** è una sequenza di stringhe $\alpha_0, \alpha_1, \ldots, \alpha_k$ t.c. $\forall 0 \leq i < k$:

1. $\alpha_i = \gamma_i A_i \delta_i$;
2. $A_i \to \theta_i \in P$;
3. $\alpha_{i+1} = \gamma_i \theta_i \delta_i$.

In tal caso diciamo che α_k è derivabile da α_0, $\alpha_0 \overset{*}{\Rightarrow} \alpha_k$.

Definizione 2.3: Linguaggio di una CFG

Data una CFG $G = (T, N, P, S)$, il suo **linguaggio** $L(G)$ è definito come:

$$L(G) = \{u \in T^* \mid S \overset{*}{\Rightarrow} u\}$$

2.1.2 Parse Tree

Definizione 2.4: Parse Tree

Data una CFG $G = (T, N, P, S)$ un **parse tree** in G è un **albero radicato** tale che:

- la radice è S;
- ogni foglia è un terminale o ϵ;
- Ogni nodo interno è un non-terminale;
- Se A è un nodo interno con figli $X_1, \ldots, X_n$ allora $A \to X_1 \ldots X_n \in P$.

Teorema 2.1: Corrispondenza tra parse tree e $L(G)$

Siano $G = (T, N, P, S)$ una CFG, T un parse tree in G e u una stringa:

$$u \in L(G) \iff \exists \text{ PT per } u$$

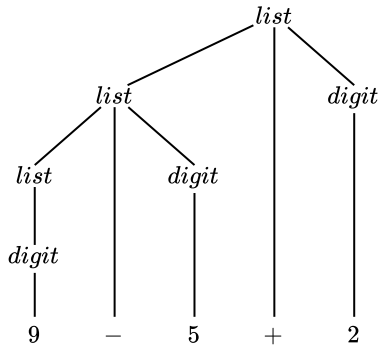
In particolare, T è una parse tree per u se le foglie di T concatenate da sinistra a destra formano u.

Esempio:

$$list \to list + digit \mid list - digit \mid digit$$
$$digit \to 0 \mid 1 \mid \cdots \mid 9$$

Derivazione per $9 - 5 + 2$:

1. $list$
2. $list + digit$
3. $list - digit + digit$
4. $digit - digit + digit$
5. $9 - digit + digit$
6. $9 - 5 + digit$
7. $9 - 5 + 2$



Data una CFG G ed una stringa u di simboli terminali, **fare il parsing** di u significa stabilire se $u \in L(G)$, ma le CFG non sono adatte a verificare ciò in modo efficiente.

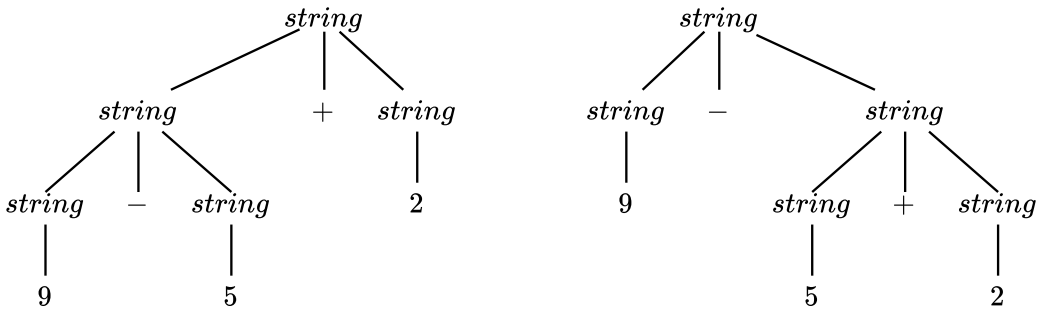
Definizione 2.5: Ambiguità di una CFG

Una CFG $G = (T, N, P, S)$ **è ambigua** se esiste $u \in L(G)$ per cui esistono due parse tree.
G **non è necessariamente ambigua** se esiste $u \in L(G)$ per cui esistono due derivazioni da S a u.

Esempi:

- *CFG ambigua:*

$$string \to string + string \mid string - string \mid 0 \mid 1 \mid \cdots \mid 9$$



- *CFG non ambigua:*

$$list \to list + digit \mid list - digit \mid digit$$

$$digit \to 0 \mid 1 \mid \cdots \mid 9$$

1° derivazione per $9 - 5 + 2$:

1. $list$
2. $list + digit$
3. $list - digit + digit$
4. $digit - digit + digit$
5. $9 - digit + digit$
6. $9 - 5 + digit$
7. $9 - 5 + 2$

2° derivazione per $9 - 5 + 2$:

1. $list$
2. $list + digit$
3. $list + 2$
4. $list - digit + 2$
5. $list - 5 + 2$
6. $digit - 5 + 2$
7. $9 - 5 + 2$

Definizione 2.6: Associatività

Un operatore è **associativo a sinistra** se $a \circ b \circ c$ è interpretato come $(a \circ b) \circ c$, mentre è **associativo a destra** se è interpretato come $a \circ (b \circ c)$.
L'associatività deve essere trattata a livello di grammatica.

Per gestire l'associatività, la grammatica deve essere strutturata con ricorsione a sinistra (per operatori associativi a sinistra) o a destra (per operatori associativi a destra):

- $list \to list + digit \mid list - digit \mid digit$ **(associatività a sinistra)**
  $digit \to 0 \mid 1 \mid \cdots \mid 9$
- $right \to letter = right \mid letter$ **(associatività a destra)**
  $letter \to a \mid b \mid \cdots \mid z$

Definizione 2.7: Precedenza

Operatori differenti avranno, in generale, precedenze differenti.
Le precedenze devono essere trattate a livello di grammatica.

Per gestire le precedenze a livello di grammatica, si parte definendo la lista delle precedenze in ordine crescente e mettendo sulla stessa riga gli elementi con la stessa precedenza, ad esempio:

- $+ \; -$
- $* \; /$
- $digit \; ()$

Successivamente per ogni livello di precedenza, a partire dal maggiore, si crea un nuovo non terminale con le relative produzioni "collegandole" al non terminale creato in precedenza, ad esempio:

$$E \to E + T \mid E - T \mid T$$

$$T \to T * F \mid T/F \mid F$$

$$F \to digit() \mid (E)$$

2.2 Traduzione

Per questa fase assumeremo di non aver bisogno del lexer e di avere un parser funzionante.

Definizione 2.8: Notazione postfissa

La **notazione postfissa** è una rappresentazione lineare di un'espressione in cui l'operatore segue i suoi operandi. Per un'espressione E può essere definita induttivamente:

1. E è una variabile o una costante $\implies post(E) = E$;
2. $E = E_1 \circ E_2 \implies post(E) = post(E_1)\; post(E_2)\; \circ$;
3. $E = (E_1) \implies post(E) = post(E_1)$.

Il nostro obiettivo sarà quello di definire un **traduttore** che, data un'espressione aritmetica E, produca $post(E)$. Per farlo useremo due approcci: **Syntax-Directed Semantic Definitions** (dichiarativo) e **Syntax-Directed Translation Schemes** (imperativo).
Considerato che in una produzione lo stesso simbolo può comparire più volte, data $A \to \alpha$ e dato un simbolo X in α, **numereremo** le diverse occorrenze di X in α in maniera crescente da sinistra a destra, così da distinguerle. Ad esempio:

$$expr \to expr_1 + term \mid expr_1 - term \mid term$$

$$term \to 0 \mid 1 \mid \cdots \mid 9$$

2.2.1 Syntax-Directed Semantic Definitions

Definizione 2.9: Syntax-Directed Semantic Definitions

Data una CFG $G = (T, N, P, S)$, una **Syntax-Directed Semantic Definitions** per G:

- associa ad ogni $X \in N \cup T$ un **insieme di attributi**;
- associa ad ogni $A \to \alpha \in P$ un **insieme di regole semantiche** che definiscono i valori degli attributi di A e dei simboli in α.

L'attributo p di un simbolo A è indicato con $A.p$.

Definizione 2.10: Regola semantica

Una **regola semantica** per una produzione $\mathcal{P} = A \to \alpha$ ha la forma $X.p = f(\mathcal{P})$ dove:

- X è un simbolo presente in $\mathcal{P}$;
- f è un'espressione contenente attributi di simboli in $\mathcal{P}$.

Definizione 2.11: Attributi sintetizzati ed ereditati

Un attributo $X.p$ è **sintetizzato** se per ogni regola $X.p = f(A \to \alpha)$, $X = A$.
Un attributo $X.p$ è **ereditato** se esiste una regola $X.p = f(A \to \alpha)$ t.c. $X \in \alpha$.

Esempio:

Produzioni	**Regole Semantiche**
$expr \to expr_1 + term$	$expr.t = expr_1.t \parallel term.t \parallel$ '+'
$expr \to expr_1 - term$	$expr.t = expr_1.t \parallel term.t \parallel$ '-'
$expr \to term$	$expr.t = term.t$
$term \to 0$	$term.t =$ '0'
$\vdots$	$\vdots$
$term \to 9$	$term.t =$ '9'

In questo esempio, ogni simbolo non terminale ha un solo attributo e ogni produzione ha una sola regola semantica. Inoltre, tutti gli attributi sono sintetizzati e t è una stringa, con $\parallel$ che indica la concatenazione.

Gli attributi sintetizzati possono essere **calcolati in modo efficiente** tramite una visita in profondità del parse tree oppure "al volo" durante il parsing. Gli attributi ereditati, invece, richiedono una **gestione più complessa**, in quanto, anche avendo il parse tree, una visita in profondità non funzionerebbe e la traduzione "al volo" durante il parsing non sarebbe sempre possibile.

2.2.2 Syntax-Directed Translation Scheme

Definizione 2.12: Syntax-Directed Translation Scheme

Data una CFG $G = (T, N, P, S)$, un **Syntax-Directed Translation Scheme** per G aggiunge ad ogni $A \to \alpha \in P$ una serie di **azioni semantiche** $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_k$, ottenendo la produzione

$$A \to \alpha_0 \mathcal{C}_1 \alpha_1 \mathcal{C}_2 \alpha_2 \ldots \mathcal{C}_k \alpha_k$$

dove ogni α_i può essere ϵ e $\alpha_0 \alpha_1 \ldots \alpha_k = \alpha$.

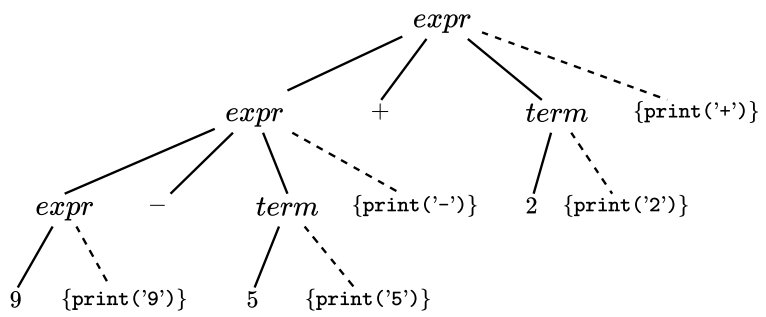
Definizione 2.13: Azione semantica

Un'**azione semantica** ha la forma {`frag`}, dove `frag` è un **frammento di codice** tipicamente scritto nel linguaggio d'implementazione del traduttore.

Esempio:

$$expr \to expr + term \ \{\texttt{print('+')}\}$$
$$expr \to expr - term \ \{\texttt{print('-')}\}$$
$$expr \to term$$
$$term \to 0 \ \{\texttt{print('0')}\}$$
$$\vdots$$
$$term \to 9 \ \{\texttt{print('9')}\}$$

Consideriamo le azioni semantiche come **foglie extra** del parse tree:



Assumendo di aver costruito il parse tree con le foglie extra per rappresentare le azioni semantiche, l'ordine di esecuzione di tali azioni corrisponde a quello di una **Depth First Visit**. Qualora il parser possa garantire lo stesso ordine di esecuzione, allora è possibile eseguire le azioni semantiche "al volo".

Una definizione semantica è associata ad una produzione, mentre un'azione semantica è associata ad una produzione e ad una particolare posizione all'interno di essa. Ciò rende le azioni semantiche **più potenti**, ma le SDSD hanno la possibilità di avere **attributi ereditati**, di conseguenza possiamo concludere che SDSD $\equiv$ SDTS.

2.3 Parsing

Per questa fase assumeremo di avere un lexer funzionante, l'input arriverà quindi già tokenizzato.
In generale per ogni CFG esiste un parser operante in $O(n^3)$, ma fortunatamente esistono due classi di CFG che permettono il parsing in $O(n)$:

- Grammatiche **LL(k)** (Left-to-right, Leftmost derivation);
- Grammatiche **LR(k)** (Left-ro-right, Rightmost derivation).

Le prestazioni del parser dipendono dal tipo di grammatica su cui si usa.

2.3.1 Bottom-Up parsing

Definizione 2.14: Parsing bottom-up

Il **parsing bottom-up** costruisce il parse tree partendo dalle foglie verso la radice. Per farlo, opera su una stringa $\mathcal{I}$, il cui valore iniziale è il testo da analizzare:

1. Individua una sottostringa α di $\mathcal{I}$ t.c. $\exists A \to \alpha \in P$;
2. Sostituisce α con A;
3. Se $\mathcal{I} = S$, termina con successo, altrimenti riparte dal passo 1;

Il limite critico di questo approccio risiede nel **non-determinismo** del passo 1: all'interno della stringa I potrebbero infatti esserci più sottostringhe α valide per la sostituzione. Per delle CFG generiche, questa ambiguità può portare il sistema a ricorrere al **backtracking**, portando la complessità computazionale a $O(n^3)$.
Nel caso delle grammatiche **LR(k)**, invece, il passo 1 arriva sempre ad un'unica scelta possibile oppure al parsing error, garantendo il determinismo ed eliminando il backtracking. Grazie a questa proprietà, il parser può operare in tempo $O(n)$.
Poiché la classe delle grammatiche **LR(k)** è più ampia di **LL(k)** ($LL(k) \subset LR(k)$), i parser bottom-up risultano particolarmente complessi. Per questo motivo, la loro costruzione richiede tool appositi (YACC ...), a differenza dei parser per grammatiche **LL(k)** che possono essere facilmente implementati a mano.

2.3.2 Top-Down parsing

Definizione 2.15: Top-Down parsing

Il **parsing top-down** costruisce il parse tree partendo dalla radice verso le foglie. Per farlo, opera su una stringa $\mathcal{I}$, il cui valore iniziale è S:

1. Individua un simbolo non-terminale A all'interno di $\mathcal{I}$ t.c. $\exists A \to \alpha \in P$;
2. Sostituisce A con α;
3. Confronta i simboli terminali ottenuti con i token in input, se coincidono avanza nell'analisi della stringa di input, altrimenti fa backtracking;
4. Ripete finché $\mathcal{I}$ non è composta da soli terminali, a quel punto il parsing termina con successo se $\mathcal{I}$ equivale all'input.

Definizione 2.16: Predictive parsing

Il **predictive parsing** è un tipo di parsing top-down applicabile quando il lookahead è sufficiente a determinare in maniera univoca quale produzione scegliere per espandere il parse tree.
Più precisamente, definito $\forall$ stringa α, $First(\alpha) = \{u \in T \mid \exists \beta \ \alpha \overset{*}{\Rightarrow} u\beta\}$, il predictive parsing è possibile se per ogni coppia $A \to \beta$ e $A \to \gamma$ si ha che $First(\beta) \cap First(\gamma) = \emptyset$.

Nel pratico, dati un lookahead t e un nodo non-terminale A nel parse tree in costruzione, il predictive parsing opera come segue:

1. Se esiste $A \to \alpha$ t.c. $t \in First(\alpha)$, il nodo A viene espanso con α;
2. Altrimenti, se $A \to \epsilon \in P$, il nodo A viene espanso con ϵ;
3. Altrimenti, **parsing error**.

Esempio:

$$
\begin{array}{rcl}
\textit{stmt} & \to & \textbf{expr} \ ; \\
 & | & \textbf{if} \ (\ \textbf{expr} \ ) \ \textit{stmt} \\
 & | & \textbf{for} \ (\ \textit{optexpr} \ ; \ \textit{optexpr} \ ; \ \textit{optexpr} \ ) \ \textit{stmt} \\
 & | & \textbf{other} \\
\textit{optexpr} & \to & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

Calcoliamo ora tutti i $First()$:

- $First(\textbf{expr} \ ;) = \textbf{expr}$
- $First(\textbf{if} \ (\ \textbf{expr} \ ) \ \textit{stmt}) = \textbf{if}$
- $First(\textbf{for} \ (\ \textit{optexpr} \ ; \ \textit{optexpr} \ ; \ \textit{optexpr} \ ) \ \textit{stmt}) = \textbf{for}$
- $First(\textbf{other}) = \textbf{other}$
- $First(\textbf{expr}) = \textbf{expr}$

Concludiamo che questa grammatica è adatta al predictive parsing.

Definizione 2.17: Recursive-descent

La **recursive-descent** è una tecnica di implementazione del predictive parsing che si basa sulla **ricorsione** messa a disposizione da un **linguaggio ad alto livello**, sostituendo così l'uso di un automa.

In particolare, il recursive-descent prevede che per ogni $A \in N$ si costruisca una **procedura** `void A()`:

1. `A` stabilisce se esista o meno $A \to \alpha$ t.c. `lookahead` $\in First(\alpha)$;
2. Se non esiste, restituisce syntax error;
3. Se esiste, allora sappiamo che è unica (per ipotesi, la grammatica si presta al predictive parsing);
4. Sia $\alpha = X_1 \dots X_k$, per ogni X_i:
   - $X_i \in T$: se è un match col `lookahead` corrente, allora `lookahead++`, altrimenti syntax error;
   - $X_i \in N$: la corrispondente procedura `void Xi()` viene invocata.

Il parser inizierà invocando `void S()`.

Esempio:

$$
\begin{array}{rcl}
\textit{stmt} & \to & \textbf{expr}\ ; \\
 & | & \textbf{if}\ (\ \textbf{expr}\ )\ \textit{stmt} \\
 & | & \textbf{for}\ (\ \textit{optexpr}\ ;\ \textit{optexpr}\ ;\ \textit{optexpr}\ )\ \textit{stmt} \\
 & | & \textbf{other} \\
\textit{optexpr} & \to & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

```
void stmt() {
    switch(lookahead) {
        case expr:
            match(expr); match(';');
            break;
        case if:
            match(if); match('('); match(expr); match(')'); stmt();
            break;
        case for:
            match(for); match('('); optexpr(); match(';'); optexpr();
            match(';'); optexpr(); match(')'); stmt();
            break;
        case other:
            match(other);
            break;
        default:
            report("syntax error");
    }
}

void optexpr() {
    if(lookahead == expr) match(expr);
}

void match(terminal t) {
    if(lookahead == t) lookahead = nextTerminal;
    else report("syntax error");
}
```

In alcuni casi può essere necessario modificare la grammatica per adattarla alla recursive-descent:

$$expr \to expr + term \mid expr - term \mid term$$

$$term \to 0 \mid \cdots \mid 9$$

Il primo problema in questo caso è che:

- $First(expr + term) = \{0, \ldots, 9\}$
- $First(expr + term) = \{0, \ldots, 9\}$

Ma ciò è facilmente risolvibile mediante **fattorizzazione**.

$$expr \to expr\ rest \mid term$$

$$rest \to +term \mid -term$$

$$term \to 0 \mid \cdots \mid 9$$

Tuttavia si presenta un secondo problema, dovuto alla ricorsione sinistra:

- $First(expr\ rest) = \{0, \ldots, 9\}$
- $First(team) = \{0, \ldots, 9\}$

Ricorsione sinistra	**Soluzione**
$A \to A\alpha \mid \beta$	$A \to \beta R$ $R \to \alpha R \mid \epsilon$

Definiamo $\alpha = rest$ e $\beta = term$ e otteniamo:

$$expr \to term\ R$$

$$R \to rest\ R \mid \epsilon$$

$$rest \to +term \mid -term$$

$$term \to 0 \mid \cdots \mid 9$$

2.4 Parsing + Traduzione

Definizione 2.18: Abstract Syntax Tree

Un **Abstract Syntax Tree** è la forma più minimalista per rappresentare, mediante un albero, un'espressione o un qualsiasi costrutto in cui esistano **operandi** e **operatori**.

In un Abstract Syntax Tree ogni **nodo interno** è un **operatore**, i cui **operandi** sono i **figli** del nodo stesso (le **foglie** dell'albero saranno sempre **operandi**).